

Agilex 5 Systolic Array Debug and Root Cause Analysis

Outcome and scope

The Linux host now reaches the 16 by 16 signed INT8 systolic array through the lightweight HPS to FPGA bridge, transfers operands and INT32 results through DMA and reserved DDR, and produces exact CPU reference matches. Single tile execution, repeated single tile execution, accumulation with padded edges, and 64 cubed and 128 cubed benchmarks passed on the board.

Correctness is established for those tests. End to end performance remains below the CPU reference because host control, memory access, resets, and completion waits dominate the measured operation. No FPGA speedup over an optimized CPU library has been established.

This analysis documents the symptoms, diagnostic evidence, root causes, corrective changes, and validation results for accelerator bring-up. Reproducible programming, boot, and deployment procedures are included. Findings established through source inspection are distinguished from failures observed on the board.

Method and evidence

The investigation used staged root cause analysis and fault isolation. Each test exposed one more part of the path while retaining previously verified stages. The sequence was CSR access, CPU access to reserved DDR, input DMA arrival, single tile arithmetic, repeated execution, accumulation and edge handling, then larger matrix correctness and timing.

The method reduced the search space. A successful CSR read did not prove DMA access; CPU memory readback did not prove FPGA DDR access; FIFO arrival did not prove operand order; capture completion did not prove output DMA completion. Some fixes were grouped after source inspection, so this was not a strict experiment changing exactly one variable at every step. The byte order diagnosis did have a strong numerical discriminator: it reproduced all eight examined mismatch values.

Stage	Board evidence	What it established
CSR diagnostics	VERSION 0x01000000 and CAPABILITY 0x01200810	Correct CSR address and accessible bridge
Reserved memory	Both 1024 byte CPU readbacks passed	CPU access to both reserved buffers
Input DMA	64 bytes arrived as 16 FIFO words	Input DMA transfer arrival
Single tile	All 256 signed outputs matched	Operand order and complete output path
Accumulation	All 323 outputs matched for 17 by 33 by 19	Three K steps and four padded output tiles
Larger matrices	4096 and 16384 outputs matched	Correct 64 cubed and 128 cubed tiled execution

Address and memory failures

CSR mapping used the wrong physical region

Symptom. The initial host configuration did not address the accelerator CSR region correctly. The host bridge base was set to 0x80000000.

Investigation. U Boot reads located VERSION at physical address 0x20020020. Linux diagnostics subsequently returned VERSION 0x01000000 and CAPABILITY 0x01200810 at the same location.

Root cause. The host used a DDR address as the lightweight HPS to FPGA bridge base. The accelerator CSR address is formed from bridge base 0x20000000 and offset 0x00020000.

Resolution. The C++ mapping was changed to use CSR base 0x20020000. CPU register addresses and DMA buffer addresses were treated separately. Address 0x38000000 was excluded as a candidate CPU DDR buffer for this configuration.

Validation. U Boot and Linux returned matching version and capability values. This establishes CSR accessibility; it does not attribute every earlier SError exception to a specific transaction because complete fault records are unavailable.

The original DDR reservation belonged to firmware

Symptom. The existing reserved DDR range at 0x80000000 through 0x81ffffff appeared available for accelerator buffers, but its ownership had not been established.

Investigation. The live device tree identified `svcbuffer@0` with base 0x80000000, size 0x02000000, `shared-dma-pool` compatibility, and `no-map`.

Root cause. This range belongs to the firmware service buffer and cannot be assigned to accelerator DMA.

Resolution. A separate 16 MiB `no-map` reservation was created at 0xb8000000 through 0xb8ffffff. Input and output tile buffers use 0xb8000000 and 0xb8010000. The driver checks the named device tree node and `/proc/iomem` before mapping the buffers.

Validation. Linux reported the new range as reserved, with System RAM on either side. Both buffers passed 1024-byte CPU write and readback tests.

DDR and DMA path validation

CPU access and FPGA DMA access were separated

The memory test wrote and read 1024 bytes at each buffer through volatile accesses and memory barriers. Both passed. The input DMA test then submitted only 64 bytes, less than a complete 128 word compute transaction. DMA became idle with its descriptor consumed and 16 input FIFO words present. That proved transfer arrival without starting matrix computation or claiming operand contents were correct.

Source inspection traced DMA memory access through the address span extender and F2SDRAM adapter to the HPS FPGA to SDRAM interface. The staged board tests provided the practical confirmation. Empty `/sys/class/fpga_bridge` meant Linux had no registered bridge devices there; it did not override the successful CSR access evidence.

Purpose	Physical address
Array CSR base	0x20020000
VERSION and CAPABILITY	0x20020020 and 0x20020024
Output and input DMA CSR	0x20020080 and 0x200200a0
Output and input descriptor ports	0x200200c0 and 0x200200d0
Reserved input and output DDR	0xb8000000 and 0xb8010000

Boot and programming issues

Programming the FPGA and booting Linux are separate steps

The combined SOF with HPS boot content is the image programmed onto the board. The packaging batch file combines the already compiled FPGA SOF with HPS SPL content; it is not itself a Quartus compilation or a board programming operation. During these C++ fixes the existing combined image could be reused. No hardware rebuild was needed for an executable change.

JTAG configuration is volatile in this workflow. After a board power cycle, program the combined image again. The saved sa_boot helper enables bridges, loads Linux, reserves DDR in the device tree, and starts Linux; it does not program the FPGA.

The original boot script enabled bridges but did not reserve DMA memory

Symptom. An interactive device tree reservation did not persist through the normal Linux boot path.

Investigation. The boot script enabled bridges and loaded Image and the SD device tree, but did not add an accelerator DDR reservation.

Root cause. Loading the unmodified SD device tree replaces the copy edited interactively in RAM.

Resolution. The sa_boot sequence reloads the device tree, creates sa_dma@b8000000, and directly boots the modified copy using booti. Loading a fresh tree also prevents duplicate node creation. The helpers are persisted with saveenv; bootcmd remains unchanged.

Validation. saveenv completed successfully. After run sa_boot, Linux exposed the reservation in /proc/iomem and the live device tree. The helper must run at each boot; saving environment variables does not modify the SD DTB.

Command and tool errors were resolved

run distro bootcmd failed because U Boot treated distro as the environment variable name. The variable is distro_bootcmd; the valid form is run distro_bootcmd. Once reservation helpers are in use, run sa_boot instead for this accelerator workflow.

A bare setenv printed usage, and savenv was an unrecognized spelling. saveenv succeeded after the helper definitions.

dtc was not installed, so decompiling the running tree failed. Direct reads of device tree files and U Boot fdt print provided the needed evidence without installing tools.

strings was also unavailable. dd exposed readable script text, but the file was a FIT container. The attempted fixed 72 byte skip was not a validated FIT parser or an extraction method to reuse.

A sed command failed because /tmp/running.dts had never been created. This was a consequence of missing dtc rather than a device tree failure.

Reservation persistence was verified after Linux boot

The successful boot showed b8000000-b8ffffff : reserved in /proc/iomem. The live node had no-map and a 16 byte reg property encoding base 0xb8000000 and size 0x01000000. Saving the helper variables alone does not edit the SD DTB; running sa_boot applies the reservation again at each boot.

Numerical failure and byte order resolution

Symptom. The first full tile test completed both DMA transfers and drained the FIFOs, but all 256 output elements differed from the reference. Expected leading values were 16, -21, 8, -29, 33, -26, 3, and -1; returned words included -50331649, 67108864, and 469762048.

Investigation. Output byte swapping alone did not recover the reference. Inspection of the generated MSGDMA read and write masters identified reversal of 8-bit symbols between memory mapped words and Avalon streaming order. Reversing each four-byte operand group and applying the output conversion reproduced all eight examined mismatch values.

Root cause. The host packed operand element zero into CPU bits [7:0] based on the array loader order, without accounting for MSGDMA symbol reversal before the loader. Output INT32 words also required decoding after DMA symbol reversal.

Resolution. Operand packing and output decoding were corrected in C++. The FPGA image and RTL were unchanged.

Validation. The rebuilt single-tile test matched all 256 signed results, and repeated execution passed. The implementation and observed status values are detailed below.

Host code correction

The CPU now packs the first INT8 element into bits 31 through 24. Input MSGDMA reverses the symbols and presents that element to the array in bits 7 through 0. Output MSGDMA reverses each INT32 word in the other direction; the host swaps its four bytes before comparing the signed result. memcpy preserves the decoded signed bit pattern without relying on an out of range unsigned to signed cast.

```
// Input word uses CPU bits 31:24 for element zero.
word |= uint32_t(uint8_t(value[lane])) << ((3 - lane) * 8);

// Decode each output word after output MSGDMA symbol reversal.
decoded = ((raw & 0x000000ffU) << 24)
| ((raw & 0x0000ff00U) << 8)
| ((raw & 0x00ff0000U) >> 8)
| ((raw & 0xff000000U) >> 24);
std::memcpy(&actual, &decoded, sizeof(actual));
```

Board resolution

The rebuilt --tile-test reported PASS: all 256 signed results match the CPU reference. Repeated execution also passed. Final array status 0x14 represents IRQ pending and output FIFO empty; it is not a claim that the DONE or input ready bits are set. Input and output DMA status 0x2 showed idle engines with descriptor FIFO empty. FIFO_STATUS 0xffff0000 showed zero FIFO levels; the reported depth fields saturate at 255 and are not the actual large FIFO capacities.

The byte order fix changed C++ only. It did not require an RTL edit, Quartus rebuild, or repackaging of a new FPGA image.

Accumulation and driver corrections

Accumulation had to begin on the first K tile

Symptom. Inspection of the initial host sequence identified an accumulation configuration that could not reliably retain the first partial result across K tiles.

Investigation. CFG_ACCUMULATE was enabled only after the first K step. With accumulation disabled, the RTL captures that step and resets the first_tile sequence.

Root cause. The host configuration treated the first K tile as an independent operation rather than the beginning of an accumulation sequence.

Resolution. CFG_ACCUMULATE is enabled on every K step, including the first. CFG_LAST_TILE and output DMA are enabled only for the final step. Accumulators clear at sequence start, retain intermediate sums, and capture the final result once.

Validation. The 17 by 33 by 19 test executed four output tiles with three K steps each and matched all 323 reference elements.

Array completion alone was insufficient

Symptom. The initial wait routine returned on IRQ pending, permitting host progress before DDR output completion was established. This defect was identified by source inspection rather than an isolated board failure.

Investigation. Array capture completion can precede output DMA completion. IRQ pending alone does not establish that output memory is ready or input storage can be reused.

Root cause. The completion condition covered the array but omitted DMA completion and descriptor state.

Resolution. Completion checks require the array indication and idle state, both DMA engines idle with descriptor FIFOs empty, and empty streaming FIFO levels. Stopped and error conditions abort execution. Monotonic deadlines bound each step to two seconds. Memory barriers protect buffer publication, descriptor submission, array start, and result access. Descriptor GO uses 0x80000000; unused address fields are zero. The earlier 0x80000003 value was removed.

Validation. Single-tile, repeated, accumulated, and larger tiled board tests passed using the revised completion condition. Fault recovery under stalled or malformed traffic remains separately unverified.

Edge tiles were padded to avoid rectangular timing

Symptom. Native rectangular execution could truncate later B columns when ACT_N exceeds ACT_M. The timing defect was identified by RTL inspection.

Investigation. The compute feed interval uses $ACT_M + ACT_K - 1$, which can be insufficient for the last B columns under those dimensions.

Root cause. The runtime feed interval does not account for the larger active output width.

Resolution. The host zero pads operands to 16 by 16, programs full array dimensions, drains all 256 output words, and copies only valid rows and columns. DMA output indexing uses stride 16. This is a host workaround; native rectangular RTL timing is not corrected by it.

Validation. The irregular 17 by 33 by 19 accumulation test matched all 323 elements through the padded path. Native rectangular execution requires a separate RTL correction and direct tests.

Implementation changes and validation controls

test/gemm_test.cpp received corrected addresses, reservation checks, diagnostics and memory modes, input DMA and tile tests, byte conversion, accumulation sequencing, padded tiling, bounded completion waits, and benchmark instrumentation.

The obsolete test/gemm_test.c implementation was removed; test/gemm_test.cpp is the maintained Linux HPS implementation.

GHRD/software/sa_reserved_boot.txt records the saved U Boot commands.

Benchmark execution stayed disabled while individual stages were unverified. It was enabled after the accumulation test passed, with initial dimensions limited to 1 through 256 and malformed arguments rejected. Subsequent validated scaling extended the supported dimension range to 1024.

The 17 by 33 by 19 accumulation test executed four output tiles with three K steps each and matched all 323 results. The later 64 cubed and 128 cubed tests matched all 4096 and 16384 results. These results validate the corrected sequence for the tested inputs, rather than every possible runtime mode.

Rebuild transfer and board run procedure

Build on the development computer

Cross compile the Linux AArch64 executable from the directory containing `gemm_test.cpp`. The Windows host does not run this ARM Linux binary; it builds and transfers it. The board runs it after booting Linux.

```
aarch64-none-linux-gnu-g++ -O3 -Wall -Wextra -std=c++17 gemm_test.cpp -o gemm_test
```

Host compilation checks establish syntax and command-line behavior. Linux memory mapping, ARM memory ordering, and DMA operation require the cross-compiled executable and board validation. Keep `gemm_metrics.hpp` and `gemm_suite.hpp` beside `gemm_test.cpp` when building the current implementation.

Replace the executable on the SD card

The SD card also holds the running Linux root filesystem. Do not remove it while the board is running. From Linux:

```
sync
sudo poweroff
```

Wait for shutdown, turn the board power off, remove the card, and copy the new executable to its FAT boot partition using the development computer. Safely eject the card, reinstall it, and power the board. Program the existing combined SOF with HPS boot content through Quartus JTAG and interrupt U Boot autoboot. Use the saved reservation helper:

```
run sa_boot
```

Log into Linux and run the test from the mounted FAT partition:

```
sudo mount /dev/mmcbk0p1 /mnt
cd /mnt
sudo ./gemm_test --diag
sudo ./gemm_test --tile-test
sudo ./gemm_test --accum-test
```

If the partition is already mounted at `/mnt`, use that mount instead of mounting it again. Updating only C++ does not require recompiling the FPGA, but powering off still requires restoring the volatile JTAG image on the next session.

Executable deployment and configuration control

Every executable revision required cross compilation, board shutdown, physical SD removal, PC file replacement, safe ejection, board power up, image programming, autoboot interruption, Linux startup, mounting, and retesting. Accidentally copying an old binary or taking the normal boot path could invalidate the experiment. The startup text identifying packing and test mode helped confirm the expected revision.

Saving `sa_boot` removed repetitive device tree editing from each cycle. Network transfer could shorten later iterations if configured; it was not established or tested during initial validation. A permanent device tree reservation was also not installed on the SD card.

Saved U Boot setup

Define each helper at the U Boot prompt. Each setenv paragraph below is one command, even where Word wraps it on the page. For direct copying, the exact commands are also saved in GHRD/software/sa_reserved_boot.txt. Single quotes defer expansion of address variables until the helper runs.

```
setenv sa_load 'mmc rescan && fatload mmc 0:1 ${kernel_addr_r} Image && fatload mmc 0:1 ${fdt_addr_r} socfpga_agilex5_de25s.dtb'
setenv sa_reserve 'fdt addr ${fdt_addr_r} && fdt resize 0x1000 && fdt mknode /reserved-memory sa_dma@b8000000 && fdt set /reserved-memory/sa_dma@b8000000 reg <0x0 0xb8000000 0x0 0x01000000> && fdt set /reserved-memory/sa_dma@b8000000 no-map && fdt print /reserved-memory/sa_dma@b8000000'
setenv sa_start 'setenv bootargs "console=ttyS0,115200 root=/dev/mmcbk0p2 rw rootwait"; booti ${kernel_addr_r} - ${fdt_addr_r}'
setenv sa_boot 'if bridge enable && run sa_load && run sa_reserve; then run sa_start; else echo SA boot aborted -- check the error above; fi'
saveenv
```

saveenv was tested successfully. After programming and interrupting autoboot on subsequent sessions, run sa_boot. No bootcmd change was made. sa_boot enables the bridge before Linux loads and aborts if bridge enable, image loading, or reservation construction fails.

Manual equivalent used during bringup

```
bridge enable
md.l 20020020 1
md.l 20020024 1
mmc rescan
fatload mmc 0:1 ${kernel_addr_r} Image
fatload mmc 0:1 ${fdt_addr_r} socfpga_agilex5_de25s.dtb
fdt addr ${fdt_addr_r}
fdt resize 0x1000
fdt print /reserved-memory
fdt mknode /reserved-memory sa_dma@b8000000
fdt set /reserved-memory/sa_dma@b8000000 reg <0x0 0xb8000000 0x0 0x01000000>
fdt set /reserved-memory/sa_dma@b8000000 no-map
fdt print /reserved-memory/sa_dma@b8000000
setenv bootargs "console=ttyS0,115200 root=/dev/mmcbk0p2 rw rootwait"
booti ${kernel_addr_r} - ${fdt_addr_r}
```

Expected CSR words are 01000000 for VERSION and 01200810 for CAPABILITY. Do not follow the reservation edit with the normal boot or distro script: that script reloads the unmodified SD device tree. Use the direct booti sequence or run sa_boot.

Linux commands and test sequence

System and reservation inspection

The following commands were used to identify the active memory layout, firmware reservation, bridge registration, and saved boot behavior.

```
cat /proc/cmdline
sudo cat /proc/iomem
ls -l /sys/class/fpga_bridge
ls -l /sys/firmware/devicetree/base/reserved-memory
sudo od -An -tx1 /sys/firmware/devicetree/base/reserved-memory/svcbuffer@0/reg
ls -l /sys/firmware/devicetree/base/reserved-memory/sa_dma@b8000000
sudo od -An -tx1 /sys/firmware/devicetree/base/reserved-memory/sa_dma@b8000000/reg
```

The new reg bytes were 00 00 00 00 b8 00 00 00 00 00 00 00 01 00 00 00. With two address cells and two size cells, they encode the 64 bit base 0xb8000000 and 64 bit size 0x01000000.

The unavailable tools and unsuccessful inspection attempts were:

```
command -v dtc
sudo dtc -I fs -O dts /sys/firmware/devicetree/base -o /tmp/running.dts
sed -n '/reserved-memory {/,/^ };/p' /tmp/running.dts
strings /mnt/boot.scr.uimg
dd if=/mnt/boot.scr.uimg bs=1 skip=72 status=none
```

dtc and strings were absent. The dd output exposed readable FIT script content but included binary metadata; it was inspection only. U Boot printenv bootcmd and printenv distro_bootcmd showed bootcmd=run distro_bootcmd and its target iteration.

Executable tests in increasing scope

```
sudo ./gemm_test --diag
sudo ./gemm_test --mem-test
sudo ./gemm_test --dma-input-test
sudo ./gemm_test --tile-test
sudo ./gemm_test --tile-test
sudo ./gemm_test --accum-test
sudo ./gemm_test --benchmark -M 64 -K 64 -N 64
sudo ./gemm_test --benchmark -M 64 -K 64 -N 64 --poll-us 0
sudo ./gemm_test --benchmark -M 128 -K 128 -N 128 --poll-us 0
```

Repeated single-tile execution passed and confirmed correct reinitialization between invocations. --poll-us 0 busy polls and occupies the host CPU while waiting. Default polling sleeps for 10 microseconds per unsuccessful check. --verbose can print tile steps but adds console overhead and was off in the larger benchmark measurements.

Performance findings and measurement limits

Board results

Workload and polling	CPU reference us	FPGA wall us	FPGA counter us	Correct outputs
17 by 33 by 19 accumulation test	19.72	2617.66	7.64	323
64 cubed with sleep polling	1113.07	12026.61	40.64	4096
64 cubed with busy polling	1113.62	9108.89	40.64	4096
128 cubed with busy polling	3361.69	45417.92	323.84	16384

All rows had zero mismatches. The accumulation test printed each K step; the larger benchmark runs did not. Busy polling reduced the measured 64 cubed wall time by 24.3 percent. These are individual runs, not averages or statistical confidence intervals.

Driver timing bucket	64 cubed busy polling us	128 cubed busy polling us
Reset and buffer initialization	3196.24	11819.65
Operand packing	854.78	5609.74
CSR and DMA submission	441.10	4137.69
Completion waits	3111.60	21168.66
Output decoding and cycle reads	1431.39	2136.79

Completion waits include device execution, DMA progress, register polling, and any sleep or scheduling delay; they are not an isolated DMA latency measurement. Reset time includes the reset routine's usleep(100), CSR accesses, and output sentinel initialization. The breakdown adds to approximately the wall time, with small unclassified loop and instrumentation costs.

How metrics were calculated

CPU time is a chrono interval around the CPU reference multiplication. FPGA wall time is a chrono interval around run_tiled_gemm. Neither interval includes Linux boot or the initial driver mapping. Wall time includes packing, output tile resets, DMA submission, waits, and decoding.

The driver sums PERF_CYCLES after each step and converts using 100 MHz, or 0.01 microseconds per cycle. In the current RTL that counter includes PE reset, compute, and flush states, but excludes operand loading and result serialization. Useful operations equal 2 times M times K times N. GOPS divides useful operations by the selected interval and one billion. Padded work is excluded from useful operation counts.

For 128 cubed, the counter derived rate is 12.95 GOPS while wall throughput is approximately 0.092 GOPS. The CPU reference finishes about 13.5 times faster end to end. The reference is a simple implementation, not an optimized BLAS or NEON benchmark. A faster core interval does not establish a complete FPGA speedup.

Baseline context and unresolved issues

Baseline architecture

The 4 by 4 baseline uses 16 processing elements at 50 MHz, signed INT8 inputs, INT32 accumulators, one operand bank, and one result snapshot. Its sequential execution does not provide hardware accumulation across K tiles or overlapping double buffering.

The baseline counter covers 44 cycles for 64 MACs, or 128 operations: 0.880 microseconds and 0.14545 GOPS. The 16 by 16 counter covers a different interval that excludes operand loading and result serialization. Ratios between these counter-derived rates do not establish application speedup. Baseline architecture and calculations are detailed in the separate architecture and baseline analysis report.

Unresolved issues and validation requirements

Remove unnecessary resets between output tiles and diagnostic output sentinel writes, then remeasure. This optimization has not yet been implemented.

Reduce host operand packing and DDR access costs and investigate overlap of transfers and computation. The present host submits steps sequentially and does not exploit the hardware's full continuous pipeline.

Fix and directly test rectangular RTL timing before enabling native edge dimensions. Current padded tiling avoids the inspected issue.

Inspect output FIFO backpressure and capture reset behavior under faults or stalls. The wrapper forwards capture valid directly to FIFO write enable, and capture is not cleared by the same soft reset path as every other block. Successful normal tests do not establish recovery behavior.

Reconcile duplicate standalone and integrated IP source copies and their differing reports before hardware changes. This was a source consistency concern, not a demonstrated cause of the successful board runs.

If a complete old design comparison is required, recover its matching integration and host test. The saved 4 by 4 SOF alone does not establish HPS compatibility or its exact source revision.

Validation records

The corrective changes were validated through CSR diagnostics, reserved DDR readback, DMA arrival, single-tile arithmetic, repeated execution, accumulated edge tiles, and larger GEMM workloads. Initial performance measurements are retained above. Subsequent repeated validation extended square dimensions through 1024; the separate validation and performance analysis report presents those distributions and scaling results.